

UNIVERSIDAD DE SEVILLA
INSTITUTO DE MICROELECTRÓNICA DE SEVILLA (IMSE-CNM-CSIC)

MÁSTER UNIVERSITARIO EN MICROELECTRÓNICA:
DISEÑO Y APLICACIONES DE SISTEMAS MICRO/NANOMÉTRICOS

**Design of a Key-Generation Cryptographic
Engine for Integrated-Circuit
Implementation: a Transistor-Level
Characterized Entropy Source with FPGA
Verification against an ESP32**

Master's Thesis

Author:
Mario García Jiménez

Supervisor:
[To be assigned]

Academic year 2025–2026

Abstract

Cryptographic keys are only as good as the entropy behind them, yet most embedded key generators are evaluated as black boxes: statistical test suites are run on their output and a pass is taken as evidence of randomness. The author's previous work on the hardware random number generator of the ESP32 showed why that is not enough — a source with low min-entropy can pass NIST SP 800-22 — and NIST itself has since rejected the suite for assessing cryptographic generators. This thesis takes the opposite route: a white-box key-generation engine whose entropy is justified from the physics of the device. A ring-oscillator entropy source is designed at transistor level and its jitter is extracted from simulation and turned, through a validated stochastic model, into a lower bound on min-entropy per bit and into the design parameters of the sampler. Around it, a fully portable digital core — SP 800-90B health tests, an AES-based conditioning and CTR_DRBG with derivation function, AES-CMAC authentication and an I2C register interface — is written once and verified three ways: against the official test vectors, in simulation on a Basys 3 FPGA acting as the verification vehicle, and against an ESP32 that recomputes every result with an independent implementation (mbedtls). The same VHDL is finally synthesized on an open 130 nm standard-cell library to give the area in gate equivalents and the path to a dedicated integrated circuit. All code, models, vectors and data are released.

Keywords true random number generator, ring oscillator, jitter, min-entropy, stochastic model, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, secure element, FPGA, ASIC, ESP32.

Resumen

Las claves criptográficas valen lo que vale la entropía que hay detrás, y sin embargo la mayoría de los generadores embebidos se evalúan como cajas negras: se pasan baterías de tests estadísticos sobre su salida y un aprobado se toma como prueba de aleatoriedad. El trabajo previo del autor sobre el generador hardware del ESP32 mostró por qué no basta — una fuente con poca min-entropía puede pasar NIST SP 800-22 — y el propio NIST ha rechazado después esa batería para evaluar generadores criptográficos. Este TFM toma el camino contrario: un motor de generación de claves de caja blanca cuya entropía se justifica desde la física del dispositivo. Se diseña a nivel de transistor una fuente de entropía por oscilador de anillo, se extrae su jitter por simulación y se convierte, mediante un modelo estocástico validado, en una cota inferior de min-entropía por bit y en los parámetros de diseño del muestreador. A su alrededor, un núcleo digital totalmente portable — tests de salud SP 800-90B, acondicionado y CTR_DRBG con función de derivación basados en AES, autenticación AES-CMAC e interfaz de registros I2C — se escribe una sola vez y se verifica de tres maneras: contra los vectores oficiales, en simulación sobre una FPGA Basys 3 que hace de vehículo de verificación, y frente a un ESP32 que recalcula cada resultado con una implementación independiente (mbedtls). El mismo VHDL se sintetiza por último sobre una biblioteca abierta de celdas estándar de 130 nm para dar el área en puertas equivalentes y el camino hacia un circuito integrado dedicado. Todo el código, modelos, vectores y datos se publican.

Acknowledgements

To my family, my colleagues at Airzone, and the faculty of the Máster en Microelectrónica at IMSE-CNM, for their support during this work.

Contents

Abstract	1
Resumen	2
Acknowledgements	3
List of Figures	7
List of Tables	8
List of Acronyms	9
1 Introduction	11
1.1 Motivation: keys are only as good as their entropy	11
1.2 From the black-box ESP32 study to a white-box engine	12
1.3 Objectives	12
1.4 Scope and exclusions	13
1.5 Methodology and reproducibility conventions	14
1.6 Structure of the document	14
2 State of the Art	15
2.1 Ring-oscillator TRNG architectures	15
2.2 Stochastic models and entropy bounds	15
2.3 Embedded jitter measurement	15
2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B	15
2.5 Symmetric cores for constrained hardware	15
2.6 Commercial secure elements	15
2.7 From FPGA to ASIC: what changes in the entropy source	15
2.8 Gap addressed by this work	15
3 Entropy Source: from Transistor to Min-Entropy	16
3.1 The elementary ring oscillator	16
3.2 Stochastic model and the quality factor	17

3.3	Exact conditional entropy vs. the asymptotic bound	17
3.3.1	Design target	18
3.4	Embedded jitter estimator and its two corrections	18
3.5	Transistor-level bench and its resolution	19
3.6	Design target and divider selection	20
4	Digital Engine	21
4.1	Architecture and the single-primitive decision	21
4.2	AES-128 encryption core	22
4.3	AES-CMAC	22
4.4	CTR_DRBG with derivation function	23
4.5	Health tests	23
4.6	Capture path and clock-domain crossings	24
4.7	I2C register interface and command sequencer	24
5	Verification	26
5.1	Strategy: vectors, simulation, independent verifier	26
5.2	Reference models in Python	27
5.3	Testbenches and results	27
5.4	ESP32 verifier firmware	28
5.5	FPGA bring-up	29
6	Path to an Integrated Circuit	30
6.1	Standard-cell synthesis flow	30
6.2	Area per block in gate equivalents	30
6.3	Optimizations and their expected savings	31
6.4	What the ring oscillator needs in silicon	32
6.5	Cost and schedule of an academic shuttle	32
7	Results and Comparison	34
7.1	Entropy campaign on the FPGA	34
7.2	Comparison with the ESP32 generator	34
7.3	Comparison with commercial secure elements	34
7.4	Uncertainty budget	34
8	Conclusions and Future Work	35
8.1	Conclusions	35
8.2	Contributions	35
8.3	Future work	35
	Bibliography	36

A	Register Map	40
B	Official Test Vectors Used	41
C	Reproduction Guide	42

List of Figures

4.1	Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.	21
-----	---	----

List of Tables

3.1	Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].	18
3.2	Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].	19
5.1	Testbenches, what they check and their result (GHDL 6.0, VHDL-2008). . .	27
6.1	Logic synthesis on IHP SG13G2, flattened, no routing. $1\text{ GE} = 7.2576\text{ }\mu\text{m}^2$. Lower bound on area [SIMULATED].	31

List of Acronyms

AES	Advanced Encryption Standard
AIS	Anwendungshinweise und Interpretationen zum Schema (BSI)
APT	Adaptive Proportion Test
ASIC	Application-Specific Integrated Circuit
BSI	Bundesamt für Sicherheit in der Informationstechnik
CAVP	Cryptographic Algorithm Validation Program
CDC	Clock Domain Crossing
CMAC	Cipher-based Message Authentication Code
CTR_DRBG	Counter-mode Deterministic Random Bit Generator
DRBG	Deterministic Random Bit Generator
ERO	Elementary Ring Oscillator
FPGA	Field-Programmable Gate Array
GE	Gate Equivalent
I2C	Inter-Integrated Circuit
KAT	Known-Answer Test
LUT	Look-Up Table
MEMS	Micro-Electro-Mechanical System
MPW	Multi-Project Wafer
NIST	National Institute of Standards and Technology
PDK	Process Design Kit

PLL	Phase-Locked Loop
PUF	Physically Unclonable Function
RCT	Repetition Count Test
RO	Ring Oscillator
RTL	Register-Transfer Level
SPI	Serial Peripheral Interface
TRNG	True Random Number Generator
XADC	Xilinx Analog-to-Digital Converter

Chapter 1

Introduction

1.1 Motivation: keys are only as good as their entropy

Every secure connection an embedded device opens, every firmware image it authenticates and every credential it stores rests on a key, and every key rests on a source of randomness. If that source is weaker than assumed, no amount of cryptographic sophistication downstream recovers what was lost at the start: a 128-bit key drawn from a generator with effectively 40 bits of entropy is a 40-bit key. This is not a theoretical concern. Frequency injection on the supply rail of a commercial secure microcontroller collapsed its key space from 2^{64} to about 3300 [1]; a certified smart-card generator was found biased despite passing the standard statistical suites [2]; and the hardware generator of one of the most widely deployed IoT microcontrollers, the ESP32, silently returns pseudo-random values when its radio is off while still passing every black-box test [3].

The common thread is that randomness has been judged by looking at the output. Statistical batteries such as NIST SP 800-22 check whether a bit stream *looks* uniform and independent; they cannot tell a physical noise source from a deterministic generator seeded with a constant, and NIST said so formally in 2022 when it announced the revision of that publication, “rejecting its use for assessing cryptographic random number generators” [4]. The current standards — AIS 20/31 in Europe [5] and SP 800-90B in the United States [6] — ask for something different: a stochastic model of the noise source, an estimate of its min-entropy derived from that model, and health tests that watch the source itself rather than its output. That is a microelectronics problem before it is a cryptography problem, and it is the problem this thesis addresses.

1.2 From the black-box ESP32 study to a white-box engine

This work continues a previous project by the author that characterized the hardware random number generator of the ESP32 as a black box: the same peripheral was exercised under four operating conditions, its output was run through a self-validated implementation of the fifteen tests of SP 800-22, and a key-harvesting scheme combining several on-chip sources was proposed. Its central finding was methodological: the tests passed under conditions in which the entropy could not have been what the tests implied, which is exactly the blind spot described above. Two of its numbers also had to be revisited with hindsight — a temperature correlation was reported from a sensor whose resolution was a single step over the observed range, and a stability figure was inferred where it could have been measured — and both lessons are turned into rules in Section 1.5.

The natural next step was to build the generator instead of measuring someone else’s: an engine whose entropy source is designed, whose jitter is measured inside the device, whose min-entropy follows from a model, and whose keys are produced and used by primitives verified against the standards. The ESP32 changes role from subject to judge. It has its own AES and SHA accelerators and a mature cryptographic library, so it can recompute everything the engine produces with an implementation that shares nothing with it, over a real bus.

The engine is designed for silicon. Secure elements — small integrated circuits that generate, store and use keys on behalf of a host microcontroller over I2C — are a mature product category; the *transparency* of their entropy is not. Of three widely used parts, one ships with a NIST validation of its entropy source and two do not mention SP 800-90 anywhere in their documentation [7]. The FPGA in this work is therefore the verification vehicle, not the target: it lets millions of test vectors run against the design in minutes, which no transistor-level simulator can, before the same description is synthesized on standard cells.

1.3 Objectives

- O1 Design a ring-oscillator entropy source at transistor level, with enable and sampler, and simulate it with noise, stating the numerical resolution of the bench before any figure is read from it.
- O2 Extract the jitter from the simulated waveforms with an estimator that separates quantization, thermal and flicker components, validated beforehand on synthetic data of known jitter.
- O3 Turn the jitter into a min-entropy bound through a stochastic model checked against

an exact computation, and derive the sampler divider that meets the AIS 20/31 target with margin.

- O4 Write the digital engine — health tests, AES-based conditioning and CTR_DRBG, AES-CMAC and an I2C register interface — as technology-independent RTL verified against official vectors.
- O5 Verify the engine functionally on an FPGA with the ESP32 as an independent oracle: known-answer tests, cross interoperability and challenge-response.
- O6 Run an entropy campaign on real silicon (the FPGA’s rings) with the SP 800-90B estimators and compare with the ESP32 generator under the same methodology.
- O7 Synthesize the engine on an open standard-cell library and report its area in gate equivalents with the growth factors to core area.
- O8 Compare the design with the ESP32 generator, with commercial secure elements and with published circuits, with stated uncertainties.

The minimum defensible result is O1–O3 together with O5: an entropy source designed from the transistor, with its jitter extracted and turned into a bound, and a digital engine verified against an independent implementation. A negative result that is well measured closes the work as well as a positive one: if the accumulation exponent on hardware is two rather than one, flicker dominates and the divider cannot grow indefinitely, and that is a finding.

1.4 Scope and exclusions

Four things are deliberately out of scope, each for a stated reason. *Fabrication*: an academic shuttle is affordable and the engine fits in the smallest block offered, but the schedule from tape-out to packaged and characterized samples exceeds a master’s thesis; the cost and the path are documented, not executed. *Asymmetric cryptography*: an elliptic-curve multiplier is a project in itself and would consume the whole area budget; the engine provides symmetric keys and authentication. *Side-channel resistance*: the threat model is a logical, remote adversary; timing leaks are avoided by construction, but no masking or trace-based evaluation is attempted, and the institute’s own laboratory for such work is noted as the natural continuation. *Formal certification*: the criteria of AIS 20/31 and SP 800-90B and the official vectors are used throughout, and no validation is claimed.

1.5 Methodology and reproducibility conventions

Four conventions apply to every figure in this document. First, each number is labelled [MEASURED], [SIMULATED] or [ESTIMATED], and carries its uncertainty or the source it comes from. Second, **the resolution of the instrument is declared before any trend is interpreted**, whether the instrument is a sensor, an estimator or a simulator — the numerical noise floor of the SPICE bench and the quantization of a temperature sensor are treated alike. Third, quantities are measured rather than inferred whenever a direct measurement exists. Fourth, novelty is stated as “not found in the literature consulted”, never as “does not exist”; one such claim in an early draft of this work had to be corrected when a 2024 paper measuring exactly the quantity in question was found, and the correction is left visible in the working notes.

Everything is released: the VHDL and its testbenches, the Python models and analysis, the firmware, the synthesis flow, the official test vectors used and the raw data of every campaign, with the commands that reproduce each table.

1.6 Structure of the document

Chapter 2 reviews ring-oscillator generators, their stochastic models, the standards and the commercial landscape. Chapter 3 builds the chain from transistor to min-entropy and validates every link. Chapter 4 describes the digital engine and Chapter 5 its verification. Chapter 6 synthesizes the engine on standard cells and states the path and cost of an integrated circuit. Chapter 7 reports the hardware campaign and the comparisons, and Chapter 8 concludes.

Chapter 2

State of the Art

- 2.1 Ring-oscillator TRNG architectures
- 2.2 Stochastic models and entropy bounds
- 2.3 Embedded jitter measurement
- 2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B
- 2.5 Symmetric cores for constrained hardware
- 2.6 Commercial secure elements
- 2.7 From FPGA to ASIC: what changes in the entropy source
- 2.8 Gap addressed by this work

Chapter 3

Entropy Source: from Transistor to Min-Entropy

This chapter builds the chain that the rest of the thesis rests on: from the physical jitter of a ring oscillator to a defensible figure of min-entropy per output bit, and from that figure to the design parameters of the sampler. Every step is validated before it is used. The statistical estimators are validated against synthetic data of known jitter; the analytic bounds are validated against an exact numerical computation; and the transistor-level bench is validated by measuring its own resolution before anything is measured with it. All figures in this chapter are [SIMULATED] unless stated otherwise; the hardware campaign is reported in Chapter 7.

3.1 The elementary ring oscillator

The entropy source is an Elementary Ring Oscillator (ERO): a free-running ring oscillator RO1 whose output is sampled by a flip-flop clocked by a second ring oscillator RO2 divided by an integer K_D [8, 9]. Between two samples the phase of RO1 accumulates jitter for K_D periods of RO2; when the accumulated jitter is comparable to the period of RO1, the sampled bit becomes unpredictable. The ERO was chosen over higher-throughput alternatives (multi-ring XOR, coherent sampling, transition-effect rings) for one reason: it is the architecture for which a stochastic model with a provable entropy bound exists and for which an embedded jitter measurement method has been published and validated [10]. Throughput is not a requirement here — the engine consumes entropy only to seed a deterministic generator — whereas a demonstrable bound is.

A design decision that follows from the platform is that *the board oscillator is never used as a reference*. The Basys 3 clock source is a MEMS oscillator with a fractional Phase-Locked Loop (PLL) whose datasheet specifies a cycle-to-cycle jitter of up to 95 ps [ESTIMATED] [11], one to two orders of magnitude above the per-period jitter of a ring

oscillator in a 28 nm process (2–4 ps [ESTIMATED] [10]). Sampling RO1 with that clock would measure the phase noise of the PLL, a global and potentially inducible component, rather than the thermal noise of the ring. RO2 therefore samples RO1, and the 100 MHz system clock only drives the synchronous logic.

3.2 Stochastic model and the quality factor

Following Baudet *et al.* [8], the phase $\varphi(t)$ of RO1, measured in periods, is modeled as a Wiener process with drift μ and volatility σ^2 . Between two samples separated by Δt the phase advances $\nu = \mu \Delta t$ periods on average and accumulates a variance

$$Q = \sigma^2 \Delta t, \quad (3.1)$$

the *quality factor*. The output bit is $s = 1$ when the fractional phase lies in $[1/2, 1)$. Conditioned on the previous phase φ_0 , the bit is Bernoulli with

$$P(s = 1 \mid \varphi_0) = \sum_{k \in \mathbb{Z}} \left[\Phi\left(\frac{k+1-\varphi_0-\nu}{\sqrt{Q}}\right) - \Phi\left(\frac{k+\frac{1}{2}-\varphi_0-\nu}{\sqrt{Q}}\right) \right], \quad (3.2)$$

where Φ is the standard normal distribution function; this is the wrapped Gaussian evaluated exactly. Its first harmonic gives the maximum bias $\varepsilon_{\max} \approx (2/\pi) e^{-2\pi^2 Q}$, from which Baudet’s Corollary 1 yields the well-known expression

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} + O\left(e^{-6\pi^2 Q}\right). \quad (3.3)$$

3.3 Exact conditional entropy vs. the asymptotic bound

Equation (3.3) is routinely cited as a *lower bound* on entropy per bit. Read carefully, the inequality in the corollary is only that the entropy rate of the sequence is at least the entropy of one bit conditioned on the exact previous phase; the right-hand side is an asymptotic expansion *of* that conditional entropy, not a bound on it. Expanding the binary entropy in the bias, $h_2(\frac{1}{2} + \varepsilon) = 1 - \frac{1}{\ln 2} \sum_{n \geq 1} \frac{(2\varepsilon)^{2n}}{2n(2n-1)}$, and averaging over a uniform φ_0 with $\varepsilon = A \sin(2\pi\varphi)$, $A = (2/\pi)e^{-2\pi^2 Q}$, gives

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} - \frac{8}{\pi^4 \ln 2} e^{-8\pi^2 Q} - \dots \quad (3.4)$$

All omitted terms are negative, so *any* truncation overestimates the entropy; the next term decays as $e^{-8\pi^2 Q}$, not $e^{-6\pi^2 Q}$. Table 3.1 compares the exact value computed from (3.2) on a grid of 8192 phases with the first- and second-order expansions. Adding the second-order term reduces the error of (3.3) by a factor of 95 at $Q = 0.10$ and 4970 at

$Q = 0.20$, which confirms the derivation.

Table 3.1: Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].

Q	Exact	Order 2	Order 1 (Eq. 3.3)	Error of order 1
0.02	0.7013194	0.7100952	0.7345213	$+3.3 \times 10^{-2}$
0.05	0.9163001	0.9164920	0.9187783	$+2.5 \times 10^{-3}$
0.10	0.9886728	0.9886733	0.9887174	$+4.5 \times 10^{-5}$
0.15	0.9984319	0.9984319	0.9984327	$+8.5 \times 10^{-7}$
0.20	0.9997823	0.9997823	0.9997823	$+1.6 \times 10^{-8}$

In the operating region ($Q \approx 0.2$) the discrepancy is 10^{-8} and irrelevant; for an under-dimensioned generator ($Q \approx 0.02$) the truncated formula credits 3.3 % of entropy that does not exist. This thesis therefore uses the exact computation as the conservative value and cites (3.3) as what it is: an excellent asymptotic approximation for $Q \gtrsim 0.1$. The same numerical machinery shows that the min-entropy obtained from the first-harmonic bias, $H_\infty = -\log_2(\frac{1}{2} + \varepsilon_{\max})$, agrees with the exact worst-case value to better than 10^{-4} bit over $Q \in [0.05, 0.30]$.

3.3.1 Design target

AIS 20/31 version 3.0 [5] admits two alternative criteria for class PTG.2: Shannon entropy ≥ 0.9998 or min-entropy ≥ 0.98 per internal bit. Solving both by bisection on the exact entropies gives $Q \geq 0.2022$ and $Q \geq 0.2286$ respectively. The min-entropy criterion is the more demanding one and is also the quantity NIST SP 800-90B works with, so it is adopted here: **the design target is $Q \geq 0.23$, with a set point of $Q = 0.30$** ($H_\infty = 0.9951$), the margin covering the reduction of thermal jitter at low temperature, which is the standard adversarial test of Fischer and Lubicz [9].

3.4 Embedded jitter estimator and its two corrections

To choose K_D one must know σ^2 , and the method of Fischer and Lubicz [9] measures it from the bits themselves: with $K_D = 1$, the fraction c of pairs (b_j, b_{j+M}) that differ within a block of N pairs estimates the phase accumulated over M samples, and its variance across K blocks grows as $4MQ_{\text{raw}}$. Implemented literally, with a straight-line fit, the estimator *underestimates* Q_{raw} by 13 % to 38 % depending on the jitter level. Two causes were identified on synthetic data with known Q and corrected:

1. **Vertex effect.** c is not linear in the accumulated phase; it is a *triangular* wave of period one and slope ± 2 . When the deterministic phase $M\nu \bmod 1$ falls near

a vertex the distribution straddles it and the variance collapses — to 0.38 of its nominal value in one measured case. The vertex is detected from the measured mean itself ($\bar{c} \approx 0$ or 1 is a vertex, $\bar{c} \approx \frac{1}{2}$ the middle of the ramp), so distances M with $\bar{c}$ outside $[0.15, 0.85]$, or for which $3\sqrt{MQ}$ reaches the vertex, are discarded iteratively.

2. **Non-constant estimation noise.** Estimating a fraction from N pairs adds a variance that scales with $\bar{c}(1 - \bar{c})$, which changes with M . Fitting it as a constant intercept biases the slope; it is fitted instead as a second regressor, $\text{Var}(c) = Q(4M) + \kappa \bar{c}(1 - \bar{c})/N$.

After both corrections the estimator recovers the injected Q_{raw} within $\pm 2\%$ over a range of $150\times$ (Table 3.2), with a bias of -0.6% and a dispersion of 4.0% over eight independent seeds for $K = 20\,000$ blocks. The fitted accumulation exponent is 1.009 for a simulator without flicker noise; measured on hardware, the same exponent separates the thermal component ($b = 1$, which counts as entropy) from flicker and global jitter ($b = 2$, which does not).

Table 3.2: Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].

Q_{raw} injected	Estimated	Error	M values used
2.0×10^{-7}	1.979×10^{-7}	-1.07%	19
5.0×10^{-7}	5.060×10^{-7}	$+1.21\%$	18
2.0×10^{-6}	2.037×10^{-6}	$+1.86\%$	16
8.0×10^{-6}	8.039×10^{-6}	$+0.48\%$	9
3.0×10^{-5}	2.977×10^{-5}	-0.78%	5

A practical consequence for the hardware is that the blocks need not be contiguous: each requires only $N + M$ consecutive bits, and independent blocks give a slightly better estimate (-0.30% error with 20 000 independent chunks). The capture buffer in the FPGA therefore holds a single chunk of a few kilobits, filled at ring speed and read out slowly, instead of megabits of contiguous samples.

3.5 Transistor-level bench and its resolution

The entropy source is the only block of the engine tied to the technology, and in an integrated circuit it is designed at transistor level. A SPICE bench was built around a five-stage ring with an enabling NAND gate, with every technology-dependent parameter (device models, widths, minimum length, supply) confined to a header so that moving from the generic placeholder models used here to a foundry Process Design Kit (PDK) changes one block of the netlist. Jitter is extracted from the threshold-crossing instants

of the output by the second difference $D(M) = t_{k+2M} - 2t_{k+M} + t_k$, i.e. the Allan variance applied to phase data, whose expectation $2M\sigma^2$ is free of the bias that a simple difference over overlapping windows carries (measured at 5%), and which is the method used by Benea *et al.* to characterize rings both in Artix-7 and in 28 nm silicon [12]. Estimates at different M are combined with inverse-variance weights proportional to the number of independent windows; without that weighting the dispersion stayed at 15% regardless of the simulated length.

The bench measures its own resolution first: with all noise sources disabled, the residual jitter is the numerical noise of the simulator. The first attempt gave 0.33 ps, of the same order as the jitter to be measured; it was entirely the start-up transient of the oscillator, and discarding the first twenty cycles brings the floor to **1.38 fs**, three orders of magnitude below the signal. Only then was noise injected. Jitter scales with the square root of the injected spectral density ($\times 2.01$ measured for $\times 4$ in density, against $\times 2.00$ expected) up to the point where the r.m.s. noise current reaches about 80% of the stage drive current; there the small-signal regime breaks down, the mean period shifts, and the scaling law fails by a factor of five. A validity criterion follows for every campaign, including those on a real PDK: keep injected noise below $\sim 40\%$ of the stage current and verify that the mean period does not move when noise is enabled. With the placeholder models the ring oscillates at 4.3 GHz, which is not physical; the numbers that enter the thesis come from the foundry models, where transient noise analysis injects the device noise itself.

3.6 Design target and divider selection

With σ/T measured per period, $Q_{\text{raw}} = (\sigma/T)^2$ and the divider follows from $K_D = \lceil Q_{\text{target}}/Q_{\text{raw}} \rceil$. For the range of jitter expected in a 28 nm process the divider lies between 3×10^4 and 5×10^5 and the throughput per ring between 1 and 20 kbit/s. The model reproduces the published design point of Fischer and Lubicz ($K_D \approx 430\,000$ for $\sigma = 5.01$ ps over $T = 7.81$ ns) [9]. Such rates are sufficient here: seeding the deterministic generator with 256 bits of full entropy requires about 512 raw bits at $H_\infty \approx 0.98$, i.e. between 0.03 and 0.5 s per seed, and the engine issues a seed only at start-up and at reseed intervals. If more throughput were ever needed, the clean route is several independent EROs, each with its own bound, rather than a multi-ring XOR whose independence cannot be demonstrated [13].

Chapter 4

Digital Engine

The digital engine turns raw bits from the entropy source into keys and uses them, and it is written so that the same Register-Transfer Level (RTL) runs on the FPGA used for verification and, later, on standard cells. Nothing in it is tied to a vendor: no primitives, no memory macros, no clock-management blocks. The only technology-dependent block of the whole design is the pair of ring oscillators of Chapter 3.

4.1 Architecture and the single-primitive decision

Figure 4.1 summarizes the data path. Raw bits leave the ERO in the clock domain of the sampling ring, cross to the system clock one bit at a time, and feed three consumers: the health tests, the seed collector of the deterministic generator, and a capture buffer used only for characterization. Keys come out of a Counter-mode Deterministic Random Bit Generator (CTR_DRBG); the current key is loaded into an AES-CMAC block that answers challenges; and an Inter-Integrated Circuit (I2C) slave exposes everything as a register map.

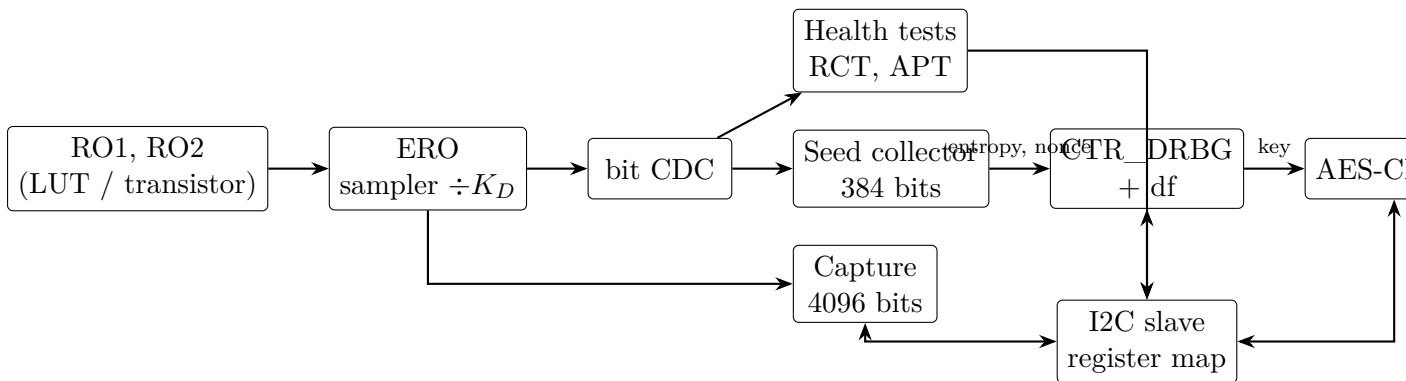


Figure 4.1: Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.

The decision that shapes the whole engine is to use **one cryptographic primitive**,

AES-128, for the three jobs: conditioning the entropy, generating keys and authenticating. The first architecture had a SHA-256 core as conditioner; the synthesis estimates of Chapter 6 showed that it would take almost half the digital area, and SP 800-90B lists AES-based constructions — CMAC, CBC-MAC and the block-cipher derivation function of SP 800-90A — among its *vetted* conditioning components [6]. Dropping the hash brings the engine from an estimated ~ 25 to ~ 13 Gate Equivalents (GEs) (thousands), at the price of moving from a CTR_DRBG *without* derivation function, which needs full-entropy input, to one *with* it, which is the common configuration in practice and accepts non-full-entropy input directly. The AES core is encryption-only: counter mode and CMAC never decrypt, and offering decryption as a service to the ESP32 would be pointless since the ESP32 has its own accelerator. Cross-verification is unaffected: the engine encrypts and the ESP32 decrypts with mbedtls.

4.2 AES-128 encryption core

The core is iterative, one round per clock cycle, eleven cycles per block (initial key addition plus ten rounds), with the round keys expanded on the fly from the previous one — forward expansion is all that encryption needs, and it saves storing eleven round keys. At 100 MHz that is 1.16 Gbit/s of core throughput, far above what any I2C link can carry, so pipelining would be wasted area; the iterative form is chosen instead because one round is a pure combinational function that can be checked directly against the standard [14]. The S-box is a 256-entry table synthesized as logic, MixColumns is written as the usual `xtime` network, and the state follows the column-major convention of FIPS 197 with byte 0 on the left, so that hexadecimal strings in the standard map one-to-one onto the VHDL vectors.

The core was verified against the five official vectors available for AES-128 encryption (FIPS 197 appendices B and C.1, and the three ECB blocks of SP 800-38A F.1.1) and against a chain of one thousand encryptions to expose any state left over between blocks. A check that had not been planned turned out to be the most convincing: the encryption of the all-zero block under the SP 800-38A example key gave `7df76b0c...546f`, which is exactly the intermediate value L published by NIST in its CMAC examples [15].

4.3 AES-CMAC

AES-CMAC [16, 17] is the primitive of the challenge-response: the ESP32 sends a nonce, the engine returns its tag under the current key. The block computes the sub-keys K_1 , K_2 once when a key is loaded (one encryption of the zero block and two conditional left shifts with the polynomial constant `0x87`) and then runs CBC-MAC over the message, applying K_1 to a complete last block or padding with `0x80` and applying K_2 otherwise.

It was chosen over HMAC-SHA-256 because it reuses the only primitive and costs one encryption per block against two hash compressions. The block passes the four vectors of RFC 4493 (empty message, one block, 40 and 64 bytes), which exercise both padding paths.

4.4 CTR_DRBG with derivation function

The generator implements CTR_DRBG with AES-128 and the block-cipher derivation function of SP 800-90A [18]. Its input format is fixed on purpose: 256 bits of raw entropy and 128 bits of nonce, so that $S = L \parallel N \parallel E \parallel \text{nonce} \parallel 0x80 \parallel 0 \dots$ is exactly four blocks and the derivation function becomes a closed sequence of fourteen encryptions; a variable-length input would cost a byte-oriented datapath for no benefit. Instantiation (or reseed) costs 14 AES operations, 183 cycles; generating 256 bits costs four, 53 cycles. The state (Key, V) is never exposed; the reseed counter is readable so that the host can enforce a reseed policy far below the 2^{48} limit of the standard.

Verification proceeded in two steps. First, a bit-exact reference model was written in Python — with its own AES, independent from the VHDL — and run against the CAVP vectors: 960 AES-128 cases with and without derivation function, with and without reseed, all passing. Second, that model generated the expected outputs for the fixed input layout of the hardware, and the VHDL reproduced them: two consecutive generations, a reseed followed by a generation, the reseed counter, and the fact that a new instantiation erases the previous state.

4.5 Health tests

The two tests approved by SP 800-90B run continuously on the raw bits [6]. The Repetition Count Test fails when $C_{\text{RCT}} = 1 + \lceil -\log_2 \alpha / H \rceil$ identical samples arrive in a row; the Adaptive Proportion Test counts, in a window of $W = 1024$ samples (the binary-source value; 512 is for non-binary sources), how many repeat the first one, and fails at $C_{\text{APT}} = 1 + \text{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$. For the design target $H = 0.98$ and $\alpha = 2^{-20}$ these are $C_{\text{RCT}} = 22$ and $C_{\text{APT}} = 596$; both are generics, because H is set from the *measured* min-entropy, not assumed. The proportion test also checks the complement of the count, which the standard permits for binary sources: a source stuck at the value opposite to the window’s first sample would otherwise score zero and pass. Alarms are sticky until explicitly cleared, and while any alarm is raised the sequencer refuses to seed.

The testbench injects the failure modes the tests exist for: a healthy source passes twenty windows (worst run 13, worst proportion 540); a stuck source trips the repetition test at exactly sample 22; a source biased to 70 % trips the proportion test at 713 without tripping the repetition test; and a source stuck at the complement is caught. The first

“healthy” source of the bench — a shift register seeded with a single one — was itself caught by the tests, with a start-up run of 27 zeros and a first window at 76 %. The detector was right and the bench was wrong; the source now starts from a dense seed and is warmed up.

4.6 Capture path and clock-domain crossings

Raw bits are produced in the clock domain of RO2 and consumed in the system domain. The bit-serial path to the health tests and the seed collector uses a toggle synchronizer: the source registers the bit and inverts a flag; the destination synchronizes the flag through three flip-flops and reads the bit on the flag’s edge. It requires each bit to last at least eight system cycles, i.e. $K_D \geq 64$ for a ~ 700 MHz sampling ring at 100 MHz, which the generator mode satisfies by orders of magnitude. The characterization path does not cross bit by bit: the capture buffer is written in the ring domain at full rate into a 4096-bit memory and read out in the system domain afterwards, with the two directions of the handshake carried by toggle flags. Writing and reading never overlap, so no asynchronous FIFO is needed, and the buffer size follows from Section 3.4: one chunk of $N + M_{\max}$ bits is all the estimator needs.

4.7 I2C register interface and command sequencer

The product interface is I2C, as in commercial secure elements: two pins, addressable, and the bus the ESP32 uses for its peripherals. The slave implements the register-pointer protocol with auto-increment, a five-sample glitch filter on both lines as the specification requires, and open-drain outputs. It does not stretch the clock: at 400 kHz there are 250 system cycles per bit, plenty for any register access, and long operations (seeding, generation) are issued as commands and polled through a status register, which is how commercial devices behave. The slave was tested with a bus-functional master; two real defects surfaced — the pointer did not auto-increment on multi-byte writes, and the most significant bit of each read byte was driven one clock edge late, so every read would have been shifted — and were fixed.

The register map (Appendix A) exposes identification, status and control, the divider and capture-page configuration, the health-test diagnostics, the reseed counter, the challenge and its tag, the last 256 bits generated and, *only while a test pin is asserted*, the current key and the capture buffer. In the FPGA the test pin is a switch; in an integrated circuit it would be a fuse blown at production, because exposing either the key or the raw entropy is a vulnerability. A small sequencer serializes the commands: seeding collects 384 raw bits, aborts if a health alarm rises meanwhile, and instantiates or reseeds the

generator; generation takes 256 bits, makes the first 128 the key and reloads the CMAC sub-keys; authentication runs the CMAC over the challenge block.

The whole engine was exercised as the ESP32 will exercise it: a bus-functional I2C master reads the identification, configures the divider, seeds, generates, reads the key in test mode, writes two challenges and reads their tags, generates again, captures raw bits and verifies that the key is unreadable without the test pin — sixteen checks. What a VHDL bench cannot know is whether the tags are *correct*, because the key came from ring-oscillator noise; the bench dumps key, challenges and tags to a file and a Python script recomputes the CMACs with the independent AES. They match. That is the same cross-check the ESP32 performs on hardware with mbedtls.

Chapter 5

Verification

5.1 Strategy: vectors, simulation, independent verifier

A cryptographic engine that is verified only against itself proves nothing: an error in the S-box would be reproduced identically in the model used to check it. The verification of this work therefore rests on three legs that do not share an implementation.

1. **Official vectors.** Every primitive is checked against the test vectors published with its standard: FIPS 197 and SP 800-38A for AES, RFC 4493 and the NIST example file for CMAC, and the CAVP response files for CTR_DRBG (with and without derivation function, with and without reseed). These are known-answer tests and they settle correctness of the algorithm.
2. **Independent reference models.** A bit-exact model of the generator was written in Python with its own AES, derived from the standard without looking at the VHDL. It is validated against the CAVP vectors first, and only then used to generate expected values for the fixed input layout of the hardware and to recompute the tags of the system-level bench. Two implementations that agree are worth more than one that passes.
3. **An independent verifier in the loop.** On hardware the ESP32 recomputes with mbedtls, and with its own AES accelerator, every result the engine produces, so that the final check is performed by a third, unrelated implementation over a real bus.

Two working rules apply throughout, both inherited from the author’s previous experimental work: the output of a compilation, simulation or measurement is never filtered before being read in full — a filter hides exactly the warning that mattered — and failures

are recorded whole, never discarded. Twice in this work a filter on the simulator’s log hid a result, and both times the fix was to remove the filter.

5.2 Reference models in Python

`ctr_drbg_ref.py` implements AES-128 encryption, the block-cipher derivation function, CBC-MAC and CTR_DRBG with and without derivation function, in pure Python and from the text of SP 800-90A. The AES asserts the two FIPS 197 vectors at import; the DRBG is exercised against the CAVP response files following the CAVS procedure (instantiate, optional reseed, two generate calls, compare the second): **960 AES-128 cases, all passing**. One defect was found in the file parser rather than in the model — a section header must close the pending test case before the parameters change, otherwise the last case of every parameter block is evaluated with the next block’s parameters — and it is worth recording because its symptom, a single failure at count 14 of each block, looked exactly like an algorithmic error.

A second script, `gen_drbg_vectores.py`, drives the validated model through the sequence the hardware supports (instantiate with 32 bytes of entropy and 16 of nonce, generate twice, reseed, generate) and emits a VHDL package of expected values; if the input layout of the hardware ever changes, that script is the only place to change it. A third, `verifica_top.py`, recomputes AES-CMAC from the same AES and validates itself against RFC 4493 before checking anything.

5.3 Testbenches and results

All VHDL is simulated with GHDL 6 under a single runner that analyses the sources in dependency order and executes every bench, printing the simulator’s output unfiltered. Table 5.1 summarizes the seven benches and their state at the time of writing.

Table 5.1: Testbenches, what they check and their result (GHDL 6.0, VHDL-2008).

Bench	Checks	P
Ring oscillator	start/stop, $f = 1/(2Nt_{pd})$, zero floor, $\sigma_T = \sqrt{2N} \sigma_{\text{stage}}$	P
I2C slave	addressing, foreign address, multi-byte write, repeated-start read	1
AES-128 core	FIPS 197 B, C.1; SP 800-38A F.1.1 (3 blocks); 1000-block chain	1
AES-CMAC	RFC 4493 examples 1–4; repeatability	1
CTR_DRBG	instantiate, 2×generate, reseed, counter, re-instantiate, refuse unseeded	1
Health tests	healthy source, stuck, 70 % biased, stuck-at-complement, sticky/clear	1
Whole engine	ESP32 sequence over I2C: id, seed, key, 2 challenges, 2nd key, capture, key hidden	1

Some of what the benches found deserves to be stated, because a verification chapter that reports only passes has not verified much.

Ring model. The LUT primitives used to simulate the ring were first modeled with a strict unknown-propagation rule: any unknown input gave an unknown output. At the start of a simulation every signal is unknown, so the ring never left that state and never started. A real NAND with its enable input at zero outputs one regardless of the other input; the model now evaluates every combination compatible with the known inputs and resolves the output when they all agree. Without this the entire system-level bench would have been impossible.

I2C slave. Two genuine design errors: the register pointer did not advance on multi-byte writes, so consecutive bytes overwrote the same address; and on reads the most significant bit was driven one clock edge too late, shifting every byte. The first was fixed by incrementing the pointer one state after the write strobe, so the address is still valid during the strobe; the second by loading the shift register and driving its first bit on the same falling edge that releases the acknowledge. A third defect was in the bench itself: initializing the register memory from the stimulus process created a second driver on the same signal and every conflicting bit resolved to unknown.

Health tests. The “healthy” source of the first bench, a shift register seeded with a single one, started with a run of 27 zeros and a first window at 76 %; the detector fired. The detector was right.

Whole engine. With the rings at their physical speed the system bench, which drives almost a millisecond of I2C traffic, did not finish in twenty minutes of CPU: every ring transition costs two random draws in the jitter model. In that bench alone the stage delay is set ten times larger, with the same relative jitter, and this is stated in the bench. The sixteen checks pass, and the key, the two challenges and their tags are written to a file that `verifica_top.py` checks: both tags equal the CMAC recomputed by the independent Python AES, the key is the first half of the generator’s output, and two generations give different keys. This is the first point at which a key born from ring-oscillator noise, conditioned, generated and used to authenticate, is confirmed by an implementation that shares nothing with the VHDL.

5.4 ESP32 verifier firmware

The firmware is an ESP-IDF 5 application for the classic ESP32 that talks to the engine through the register map of Appendix A and offers a console over the UART. Its commands map onto the verification plan: `id` reads identification and status; `sembrar` and `generar` issue the long commands and poll the busy flag; in test mode `generar` also reads back the key (provisioning) and checks that it equals the first sixteen bytes of the generated

block; `reto N` performs N challenge-response rounds with nonces from `esp_random()`, recomputing each tag with `mbedtls_cipher_cmac` and counting matches and mismatches, the mismatches being printed in full; `crudo N` dumps N chunks of raw bits and `esp N` dumps blocks of the ESP32’s own generator, both in the framed, CRC32-protected format used in the author’s previous work so that the same statistical pipeline processes the two sources. CMAC is not enabled by default in the ESP-IDF build of `mbedtls` and is switched on in the project defaults; the console runs at 115 200 baud, the only rate that proved reliable with the board’s USB bridge in earlier campaigns, and the log level is set to errors only so that log lines cannot corrupt the frames. The firmware builds with ESP-IDF 5.3.2 into a 255 kB image; running it awaits the hardware.

5.5 FPGA bring-up

Pending hardware. The constraints the rings need are known from the literature and from the vendor documentation — `DONT_TOUCH` on the cells and nets of each ring (unlike `KEEP`, it survives implementation), `ALLOW_COMBINATORIAL_LOOPS` on one net of each loop, `set_disable_timing` to break the loop for static timing analysis, and one placement block per ring so that identical rings are neither merged nor placed side by side — and the first bitstream has a single purpose: to read the frequency counters and confirm that every ring oscillates. Nothing about entropy is claimed before that.

Chapter 6

Path to an Integrated Circuit

The engine is designed to end up as a dedicated integrated circuit that serves keys to a microcontroller, and the FPGA is its verification vehicle, not its destination. This chapter takes the same VHDL that passes the testbenches of Chapter 5, synthesizes it on an open standard-cell library, and states what a fabrication run would cost and what would have to change in the entropy source. A tape-out is explicitly out of scope of this thesis — not for cost or area, as will be seen, but for schedule — and everything here is labelled as what it is: a lower bound on area from logic synthesis, with the growth factors to core area taken from published data.

6.1 Standard-cell synthesis flow

The flow is deliberately reproducible with free tools. Yosys reads the VHDL through the GHDL front end, synthesizes each block flattened, maps flip-flops and logic onto the liberty file of the target library and reports cell count and area. The library is the open PDK of IHP, SG13G2, a 130 nm BiCMOS process with a maintained standard-cell library, compiled SRAM macros, I/O cells and OpenROAD support, chosen because it is the one open PDK that is alive, European and reachable through the same Europractice membership the university already holds. The gate equivalent is read from the liberty file itself rather than from memory: 1 GE = area of `sg13g2_nand2_1` = $7.2576 \mu\text{m}^2$. No clock constraint is applied: Yosys does not use one for area, so the figures do not depend on a target frequency. The ring oscillators are not synthesized; they are the one block designed at transistor level (Chapter 3).

6.2 Area per block in gate equivalents

Table 6.1 gives the result for each block. The digital engine — everything except the capture buffer — is about **62 kGE**, and AES is 51 % of it: the authenticator and the

generator each instantiate their own encryption core, 15.8 kGE apiece. The capture buffer is reported for completeness but must not be read as part of the product: its 46.7 kGE are a 4096-bit memory mapped onto flip-flops because the flow has no SRAM macros. In the FPGA it occupies a block RAM at no logic cost; in silicon it would be a compiled SRAM or, more sensibly, a much smaller buffer, because it is a test path that is disabled at production.

Table 6.1: Logic synthesis on IHP SG13G2, flattened, no routing. 1 GE = $7.2576 \mu\text{m}^2$. Lower bound on area [SIMULATED].

Block	Cells	Flip-flops	Area (μm^2)	kGE
AES-128 encryption core	10 439	262	114 771	15.81
AES-CMAC (with its AES)	13 210	909	169 766	23.39
CTR_DRBG with df (with its AES)	17 953	2 355	267 262	36.83
Health tests (RCT, APT)	568	85	8 986	1.24
ERO sampler and divider	117	24	2 078	0.29
Bit clock-domain crossing	11	7	408	0.06
I2C slave	310	63	5 507	0.76
Capture buffer (4096-bit RAM as flip-flops)	12 250	4 140	339 057	46.72

The numbers are internally consistent, and that consistency is how an error in the flow was caught: before flattening, the report returned for the authenticator and for the generator the area of their AES sub-module alone. An authenticator weighing exactly as much as a bare AES is impossible, so the report, not the design, was wrong. After flattening, the authenticator adds to the AES 647 flip-flops (sub-keys, chaining state, key) and 7.6 kGE, and the generator adds 2093 flip-flops and 21 kGE, about half of which are its registers.

6.3 Optimizations and their expected savings

The synthesis is naive on purpose — the RTL was written to be verifiable, not small — and it points at three optimizations, in order of return on effort:

1. **One shared AES core.** The top-level sequencer already serializes commands: the generator and the authenticator never run at the same time. Sharing the core saves ≈ 15.8 kGE, a quarter of the engine, for a trivial arbiter.
2. **Compact S-box datapath.** Twenty independent S-box tables synthesize to 15.8 kGE for the whole core; area-optimized designs in the literature range from 2.4 kGE for an 8-bit datapath at 226 cycles per block [19] to about 5 kGE for 32-bit datapaths. Speed is irrelevant here, so a 32-bit datapath with four shared S-boxes is the sensible point; estimated saving, 10 kGE per core.

3. **Fewer registers in the generator.** Of its 2355 flip-flops, the captured entropy and nonce (384), the seed material (256), the output (256) and the temporaries of the derivation function can be reduced by consuming the seed as a stream and sharing temporaries; roughly 800 flip-flops, about 4 kGE.

With the three, the digital engine falls to ≈ 20 kGE, which is the order of magnitude the state of the art anticipated (13 to 25 kGE for engines of this kind) and which, at the published growth factor of ≈ 2.35 from cell area to core area for an AES on this same library, is about 0.34 mm^2 of core. Each of these savings is stated as an estimate; measuring them by synthesizing the optimized variants is listed as future work, not claimed.

6.4 What the ring oscillator needs in silicon

The digital blocks port with the standard flow. The entropy source does not, and the literature that has done the crossing — notably a 2024 study that characterizes the same ring model on an Artix-7 and on a 28 nm FD-SOI circuit [12] — makes three points that the chip design must absorb.

First, the ring is rebuilt from inverter cells rather than look-up tables, and its jitter changes in value and in character: flicker noise gains weight in newer nodes, and the study finds, against expectation, that counting it as a legitimate source can raise throughput for the same entropy. The stochastic model and the estimator of Chapter 3 carry over unchanged; the measured σ does not. Second, process variation makes every die different, so the frequency and jitter of each ring must be calibrated per chip; the published all-digital designs that survive voltage and temperature sweeps do it with a configurable ring and a tuning loop [20]. Third, and most important for a security device, injecting a frequency on the supply locks the rings and removes the jitter: a 500 mV signal on the rail reduced the key space of a commercial secure microcontroller from 2^{64} to about 3300 [1], and a few hundred microwatts of radiated field destroyed the statistics of a fifty-ring generator [21]. The same calibration loop, together with supply filtering, is the published defence, and the health tests of Section 4.5 are the detector of last resort. None of this is in the FPGA prototype; it is the work that a silicon implementation adds.

6.5 Cost and schedule of an academic shuttle

Fabrication is more accessible than the size of the word suggests. The University of Sevilla and IMSE-CNM are members of Europractice, whose academic price list is public: a mini@sic block in UMC 180 nm costs 3 430 €, one in TSMC 65 nm 3 691 €, and IHP SG13G2 is offered at 5 110 €/mm² with a free multi-project run for open-source designs under 2 mm^2 [22]. An engine of $0.3\text{--}0.4 \text{ mm}^2$ fits many times over in the smallest block

of any of them; the chip would be limited by its pads, not its logic. The institute's own hardware-security group has already taken ring-oscillator primitives from Xilinx 7-series FPGAs to characterized 65 nm silicon, which makes the continuation of this work a matter of joining an existing line rather than opening one.

What does not fit in a master's thesis is the schedule: from tape-out to packaged samples takes months to over a year, the price does not include packaging, and the characterization of the returned dies is a project in itself. For comparison, a commercial secure element such as the ATECC608B costs 0.90 \$ in single units and ships with a NIST entropy source validation [7]; two of its direct competitors do not mention SP 800-90 anywhere in their datasheets. The honest position of this work is therefore not to compete on price but on *transparency of the entropy*: every bit of a key produced by this engine can be traced to a measured jitter and a stated bound, which is precisely what a black-box part cannot offer.

Chapter 7

Results and Comparison

7.1 Entropy campaign on the FPGA

7.2 Comparison with the ESP32 generator

7.3 Comparison with commercial secure elements

7.4 Uncertainty budget

Chapter 8

Conclusions and Future Work

8.1 Conclusions

8.2 Contributions

8.3 Future work

Bibliography

- [1] A. T. Markettos and S. W. Moore, “The frequency injection attack on ring-oscillator-based true random number generators,” in *Cryptographic Hardware and Embedded Systems – CHES 2009*, ser. LNCS, vol. 5747. Springer, 2009, pp. 317–331, [VERIFICADO] PDF + Crossref; keyspace $2^{64} \rightarrow 3300$; tarjeta EMV 2004; enganche 74HC04 a 24 MHz / 900 mV pk-pk. [Online]. Available: <https://www.iacr.org/archive/ches2009/57470316/57470316.pdf>
- [2] D. Hurley-Smith and J. Hernandez-Castro, “Certifiably biased: An in-depth analysis of a common criteria EAL4+ certified TRNG,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 4, pp. 1031–1041, 2018, [VERIFICADO-META] Crossref.
- [3] J. Blanco-Romero, F. Almenares Mendoza, D. Díaz-Sánchez, and A. Marín-López, “Entropy bootstrapping for wireless embedded systems,” arXiv:2607.08865, Sep. 2026, [VERIFICADO] página abs; ESP32 RNG pseudoaleatorio con RF apagada pasa cribados estadísticos. [Online]. Available: <https://arxiv.org/abs/2607.08865>
- [4] National Institute of Standards and Technology, “Decision to revise NIST SP 800-22 Rev. 1a,” Apr. 2022, [VERIFICADO] Cita literal: “rejecting its use for assessing cryptographic random number generators”. [Online]. Available: <https://www.nist.gov/news-events/news/2022/04/decision-revise-nist-sp-800-22-rev-1a>
- [5] M. Peter and W. Schindler, “A proposal for functionality classes for random number generators, version 3.0,” Bundesamt für Sicherheit in der Informationstechnik (BSI), Mathematical-technical reference of AIS 20/31, Sep. 2024, [VERIFICADO] PDF 306 pp.; PTG.2.2: Shannon ≥ 0.9998 y/o min-entropía ≥ 0.98 , Prob(1) en (0.493,0.507); §4.5.3 online test, §4.5.4 total failure test, §4.6 suite T_{irn} . [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_31_Functionality_classes_for_random_number_generators_e_2024.pdf
- [6] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, “Recommendation for the entropy sources used for random bit generation,” NIST,

- Special Publication SP 800-90B, Jan. 2018, [VERIFICADO: PDF extraído; sección 3.1.5 completa (lista de componentes vetted, Tabla 1, fórmula Output_Entropy, factor 0,999 para no vetted). Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/b/final>
- [7] *ATECC608B CryptoAuthentication Device Summary Data Sheet*, Microchip Technology Inc., 2021, [VERIFICADO] Cita literal: "The random number generator is designed to meet the requirements documented in the NIST 800-90A, 800-90B and 800-90C documents" — redacción de intención de diseño, no de certificación. AES-128 únicamente; solo curva NIST P-256; 16 slots de clave; 1.208 B de data zone; I2C hasta 1 Mbps + SWI; sin SPI. Es un datasheet RESUMEN: el completo está bajo NDA. [Online]. Available: <https://ww1.microchip.com/downloads/aemDocuments/documents/SCBU/ProductDocuments/DataSheets/ATECC608B-CryptoAuthentication-Device-Summary-Data-Sheet-DS40002239B.pdf>
- [8] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011, [VERIFICADO] ePrint 2009/299 leído; Crossref confirma DOI/vol/págs. $Q = \sigma^2 \Delta t$; Prop. 1 ecs. 8, 10, 12; Cor. 1 ec. 14. [Online]. Available: <https://eprint.iacr.org/2009/299.pdf>
- [9] V. Fischer and D. Lubicz, "Embedded evaluation of randomness in oscillator based elementary TRNG," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 527–543, [VERIFICADO] PDF completo; Algoritmo 1; error $< 5\%$; $\sigma = 5.01$ ps / 7.81 ns en Cyclone III; $K_D \approx 430\,000$ para $H \geq 0.997$. [Online]. Available: <https://www.iacr.org/archive/ches2014/87310109/87310109.pdf>
- [10] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *26th International Conference on Field Programmable Logic and Applications (FPL 2016)*, 2016, pp. 1–10, [VERIFICADO] PDF HAL completo + Crossref; Tabla II (Spartan-6, Cyclone V, SmartFusion2); jitter 2–4 ps; ec. (1) H_{min} . [Online]. Available: <https://hal.science/hal-01382955/document>
- [11] Digilent Inc., *Basys3 FPGA Board Reference Manual (Revised August 12, 2014; applies to Basys3 rev. C)*, Digilent, 2014, xC7A35T-1CPG236C; 33 280 celdas lógicas en 5200 slices (4 LUT6 + 8 FF por slice); oscilador de 100 MHz en W5; Pmod JA/JB/JC + JXADC, 3,3 V en pines 6 y 12. Versión web actual: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>

- (no accesible en esta sesión). [VERIFICADO: PDF extraído]. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf
- [12] L. Benea, M. Carmona, V. Fischer, F. Pebay-Peyroula, and R. Wacquez, “Impact of the flicker noise on the ring oscillator-based TRNGs,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024, [VERIFICADO: PDF leído por el investigador; DOI pendiente de anotar. Datos y scripts abiertos en github.com/opentrng/papers].
- [13] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, “True-randomness and pseudo-randomness in ring oscillator-based true random number generators,” *International Journal of Reconfigurable Computing*, vol. 2010, pp. Article ID 879 281, 1–13, 2010, [VERIFICADO-META] Crossref.
- [14] National Institute of Standards and Technology, “Advanced encryption standard (AES),” NIST, Federal Information Processing Standards Publication FIPS 197-upd1, May 2023, publicado originalmente el 26-11-2001; actualización editorial (sin cambios técnicos) del 9-05-2023. [VERIFICADO: página CSRC y PDF, Tabla 3 y Apéndice D]. [Online]. Available: <https://csrc.nist.gov/pubs/fips/197/final>
- [15] —, “Block cipher modes of operation — CMAC mode for authentication: Example values (CMAC-AES128/192/256),” https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Standards-and-Guidelines/documents/examples/AES_CMAC.pdf, 2016, ejemplos retirados del Apéndice D de SP 800-38B en 2016. [VERIFICADO: PDF extraído; K1/K2 y etiquetas coinciden con RFC 4493].
- [16] M. Dworkin, “Recommendation for block cipher modes of operation: The CMAC mode for authentication,” NIST, Special Publication SP 800-38B (incl. updates as of 10-06-2016), Oct. 2016, publicado en mayo de 2005; erratas sustantivas del 6-10-2016 (frontal, retirada de ejemplos del Ap. D, bibliografía). [VERIFICADO: PDF extraído; autor según portada del PDF]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final>
- [17] J. Song, R. Poovendran, J. Lee, and T. Iwata, “The AES-CMAC algorithm,” IETF, RFC 4493, Jun. 2006, [VERIFICADO: vectores de prueba de la sección 4 confirmados; autores y fecha citados de memoria, NO VERIFICADOS en esta sesión]. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4493>
- [18] E. Barker and J. Kelsey, “Recommendation for random number generation using deterministic random bit generators,” NIST, Special Publication SP 800-90A Rev. 1, Jun. 2015, [VERIFICADO: PDF extraído; Tablas 2 y 3, secciones 8.6,

- 10.1, 10.2.1.3.1 y 11 leídas. Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/a/r1/final>
- [19] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, “Pushing the limits: A very compact and a threshold implementation of AES,” in *Advances in Cryptology — EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, vol. 6632. Springer, 2011, metadatos [VERIFICADO] vía API de Semantic Scholar; cifra de 2.400 GE [VERIFICADO] en el resumen público. AES-128 SOLO CIFRADO en librería UMC 180 nm — que es justamente lo que necesita un CTR_DRBG. [NO VERIFICADO] rango de páginas. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-20465-4_6
- [20] K. Yang, D. Blaauw, and D. Sylvester, “An all-digital edge racing true random number generator robust against PVT variations,” *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1022–1031, 2016, [VERIFICADO por el investigador: DOI y cifras de robustez PVT].
- [21] P. Bayon, L. Bossuet, A. Aubert, V. Fischer, F. Poucheret, B. Robisson, and P. Maurine, “Contactless electromagnetic active attack on ring oscillator based true random number generator,” in *Constructive Side-Channel Analysis and Secure Design (COSADE 2012)*, ser. LNCS, vol. 7275. Springer, 2012, pp. 151–166, [VERIFICADO-META] Crossref; 50 ROs enganchados por campo EM (abstract).
- [22] EUROPRACTICE IC Service, “MPW schedules and prices 2026,” <https://europactice-ic.com/schedules-prices-2026/>, 2026, [VERIFICADO] Precios públicos para UMC, IHP, X-FAB, GlobalFoundries, ams OSRAM. EXCEPCIÓN: los MPW generales de TSMC exigen formulario "Request TSMC Prices" y STMicroelectronics figura "on request": esos precios NO son públicos.

Appendix A

Register Map

Appendix B

Official Test Vectors Used

Appendix C

Reproduction Guide